

Building a universal game-playing agent infrastructure to optimize performance

Final Report

Maria Manina

Individual Module

Supervisors: Prof. David Schlangen, Dr. Sherzod Hakimov

April 30, 2026
University of Potsdam

Abstract

Large language models are increasingly deployed as autonomous agents, yet it remains unclear whether structured, agent-like augmentations are necessary for strong performance, or whether lightweight modifications to a base model are enough. We investigate this question using **clembench**, a dialogue game benchmark. In an ablation-style study, we evaluate four targeted modifications over a base LLM player and compare them against a tool-augmented LangChain agent. Our results show that most lightweight add-ons fail to improve overall performance while the LangChain Memory-Tool Agent achieves the highest results overall. Cluster-level analysis reveals that no single modification is universally beneficial: effects vary substantially across word-based, image-based, and exploration games. These findings suggest that targeted modifications offer limited gains, and that tool-augmented agent frameworks may provide a stronger foundation.

1 Introduction

There is great interest and a growing body of work in the **agentic LLMs** — large language models that behave as agents. The concept of agents is not new — (Russell et al., 1995) define agents as *anything that can be viewed as perceiving its environment through sensors and acting upon that environment through effectors*. Now we have language agents — foundational models used as agent that understand and operate primarily in the domain of human language (Manning, 2025).

Owing to their reasoning, acting and interacting abilities, language agents have been proposed for a wide range of applications: web navigation (Anthropic, 2024), code generation and debugging (Quan et al., 2025), scientific discovery (Zhao et al., 2026). One way to build a reliable agent is to use a general-purpose framework such as **LangChain**, which provides tool-calling infrastructure, memory management, and multi-step reasoning loops. However, they introduce significant architectural overhead and are more expensive to deploy and maintain.

This raises a practical question: are full agent frameworks necessary, or can lightweight modifications to a base LLM achieve comparable results? We investigate this in a controlled, multi-game setting using **clembench**¹ — a benchmark for evaluating LLMs through conversational or dialogue

gameplay. In frames of it, quite high results have been achieved (see the [clembench leaderboard](#)) — for example, Claude 3.7 Sonnet with 80.44 % of played games and a quality score of 81.85 out of 100.0.

The key distinction is that an agent is able to perform controlled actions in its own "free will", which makes it able to carry out goal-directed behaviour, while still being just a next token predictor internally (Plaat et al., 2025). Since **clembench** evaluates performance against structured, rule-governed game standards, agency may be the missing component that prevents standard LLMs from performing better. Identifying which agent-like components contribute most to improvement would enable a cost-effective, yet capable agent.

Our study tries to answer the following **research questions**: (i) *Does an LLM with some simple add-ons perform better on agentic tasks than a standard LLM (as tested by clembench)?* (ii) *What forms an optimal agent architecture that helps to maximize performance?* (iii) *Does delegating agent behaviour to a general-purpose framework (LangChain) offer an advantage over targeted, minimal modifications?*

The contributions of the paper are as follows:

- We evaluate four lightweight add-ons to a base LLM player: a Pre-Game Planning Agent, a Format-Recovery Agent, a Reflexion Agent, and an Upfront-Strategy Agent on **clembench**.
- We show that injecting human-authored game strategies yields the only consistent performance improvement among the tested add-ons, while other configurations regress.
- We evaluate a tool-augmented LangChain agent as a reference point for what general-purpose agent infrastructure achieves on the same benchmark.
- We identify game-modality effects, suggesting that no single modification is universally beneficial.
- We release all agent implementations and experiment logs in a public repository.²

²<https://github.com/mariamannina/clembench>

¹(Hakimov et al., 2025)

2 Background and Terminology

2.1 Language Agents and their components

Language agents are typically characterized by three main components: **reasoning** (forming beliefs and plans from observations), **acting** (executing operations in an environment), and **planning** (decomposing goals into sequences of steps). In practice, these components are not always implemented in isolation — frameworks such as ReAct (Yao et al., 2022) and Reflexion (Shinn et al., 2023) implement two of them jointly. Another important component is tool use. Tools are external modules that are typically called using a rule or special token and whose output is included in the augmented LLM’s context (Mialon et al., 2023). They allow agents to overcome inherent LLM limitations: arithmetic errors, outdated knowledge etc. by delegating specific operations to calculators, search engines and databases.

(Manning, 2025) argues that agents need two things to "survive": 1) enough built-in knowledge; 2) ability to learn. Vanilla LLMs satisfy the first condition only partially, and the condition is not satisfied at all: without weight updates or memory, a standard LLM cannot adapt to new tasks or correct repeated mistakes.

2.2 The clembench Framework

All the experiments were carried out using the **clembench** framework (Chalamalasetti et al., 2023). **Clembench** is a collection of text-based Dialogue Games designed to evaluate large language models across a broad range of abilities. In Dialogue Games, as defined by (Schlangen, 2023) in the context of this framework, **player(s)** attempt to reach a predetermined **goal state** primarily by means of producing and understanding linguistic material. For further details and an overview of how the framework has evolved over time, we refer the reader to subsequent papers on **clembench** (Chalamalasetti et al., 2023), (Horst et al., 2025), (Hakimov et al., 2025), (Hakimov et al., 2026). Below, we describe only the aspects that are most relevant to our research.

2.2.1 The Games Collection

Clembench v.3.0 (date of access 08.04.2026) encompasses **16** games, **13** of which were tested in the framework of this study. We group the games into three clusters for convenience, facilitating future analysis:

1. Word-based / semantic games

- codenames
- guesswhat
- taboo
- privateshared
- wordle

2. World exploration / interaction-based games

- adventuregame
- mapworld (text-based & multimodal versions)

3. Image-based games

- imagegame
- matchit (image-based and ASCII-based versions)
- referencegame (base and multimodal versions)

For short game descriptions, see the aforementioned papers and Appendix A.

2.2.2 The Game Flow

Each **clembench** game is turn-based, with strictly alternating turns and a fixed maximum number of turns. The interaction between players is managed by a programmatic *Game Master*. It tracks the game state and penalizes non-relevant actions.

On the first turn, players receive an initial prompt containing the game rules and the overall objective. Each player’s response is then passed to the Game Master, which evaluates it and, if acceptable, forwards it to the other player (if one exists). Otherwise, the Game Master returns feedback to the player, indicating how the environment has changed. All information about the game state is mediated by the Game Master; players cannot *directly* observe the environment or interact with each other. In two-player games, players typically assume distinct roles (e.g., Guesser and Describer in Taboo).

Each player response is expected to (i) strictly adhere to the required format; and (ii) contribute to achieving the game objective, with format compliance being the highest priority. **Clembench** enforces a strict response format, and even minor deviations may result in the response being disregarded. In particular, each response must begin with a game-specific tag (e.g., "Command:" in the Image game).

The game terminates when one of the following conditions is met: (i) the maximum number of turns is reached; (ii) the objective is achieved (e.g., the target word is guessed); or (iii) the game is aborted due to repeated rule violations or other game-specific conditions.

For reference, we provide an example transcript of a successful Taboo game in Appendix B.

2.2.3 Game Play Evaluation

During game play, two main aspects are evaluated: first, the general ability to follow rules, expressed as the *percentage of played* (i.e., non-aborted) *games*; second, the game-specific *quality score* quantifying the players’ competence in playing the game. The metrics are typically the same for both players despite their asymmetric roles. Together, these two form the *clemscore* — the macro-average quality score multiplied by the macro-average proportion of played games. That is, given N games, the *clemscore* is computed as follows:

$$\left(\frac{1}{N} \sum_{i=1}^N q_i\right) \left(\frac{1}{100N} \sum_{i=1}^N \%p_i\right),$$

where q_i is the quality score for the i -th game and $\%p_i$ is the percentage of played games for the i -th game.

3 Experiments

This section summarizes all tested agent architectures and highlights selected observations from the results. During analysis, we follow the principle of cluster analysis of games introduced earlier. We also follow an ablation-style design for the **clembench**-based configurations, introducing exactly one modification over the baseline per agent.

The general experimental procedure was as follows:

1. Test the baseline using the Qwen3-VL-30B-A3B-Instruct-FP8 (temperature = 0.0)
2. Add one (and only one) component to the baseline model
3. Test the resulting agent architecture using the Qwen3-VL-30B-A3B-Instruct-FP8 (temperature = 0.0)
4. Perform qualitative and quantitative analysis of the results

5. Deem the experiment as successful or unsuccessful

For each game, a new set of instances was generated, ranging from **20** to **30** per game (depending on the number of experiments within each task). Each game consists of several *experimental conditions* (e.g., word-based games with words of varying frequency, or exploration-based games with different environment sizes such as small and large rooms). A detailed analysis of performance under different experimental conditions would be of interest, but it is beyond the scope of this paper.

Therefore, our evaluation does not aim to compare against the official [clembench leaderboard](#). Instead, we focus on relative performance comparisons between a baseline configuration and its variants under identical instance sets. All agent architectures were evaluated on the **same set of generated instances** per game.

3.1 Baseline

The baseline consists of a vanilla LLM without any additional modifications. The model used for the baseline is Qwen3-VL-30B-A3B-Instruct-FP8. The results are summarized in Table 1.³

Game	clemscore	% Played	Qlty Score
adventuregame		79.17	5.26 (16.72)
codenames		38.46	30.0 (48.3)
guesswhat		0.0	NaN
imagegame		0.0	NaN
matchit		95.83	82.61 (38.76)
matchit_ascii		100.0	87.5 (33.78)
multimodal_mapworld	26.61	84.0	60.43 (16.23)
multimodal_referencegame		100.0	77.27 (42.89)
privateshared		100.0	NaN
referencegame		100.0	72.0 (45.83)
taboo		100.0	26.67 (45.77)
textmapworld		0.0	NaN
wordle		70.0	3.57 (13.36)

Table 1: Baseline metrics for Qwen3-VL-30B-A3B-Instruct-FP8. Throughout this paper, *% Played* indicates the percentage of games played to completion, *Qlty Score* shows how well the completed games were played (higher is better, max is 100; standard deviation in parentheses). NaN indicates that no episodes were successfully completed, making quality scores undefined.

In this baseline setting, performance varies substantially across game types. For exploration-based and image-based games, there are two

³For the *privateshared* game, the scoring shows 100% played. However, it is the only game that does not lead to aborts due to incorrect or missing tags. Therefore, the scoring here may not be fully reliable.

extremes: either a high % played (e.g., *multimodal_mapworld*, *referencegame*) or close to zero (*textmapworld*, *imagegame*). As for the quality score, it varies from low for *adventuregame* and *imagegame* to moderate in *multimodal_mapworld* and high in *matchit* and *multimodal_referencegame*.

As for the word-based and semantic-based games (*codenames*, *guesswhat*, *taboo*, *private-shared*, and *wordle*), this cluster exhibits the lowest quality scores overall. Combined with a high standard deviation, this indicates unstable performance across tasks within this category. *Wordle* and *taboo*, which are widely known games, also exhibit a high percentage of played episodes, whereas *codenames* shows a noticeably lower completion rate, despite being a well-known task for both humans and language models.

Overall, the experiment demonstrates stronger performance in matching tasks and one-turn games, and weaker performance in long-horizon reasoning and multi-step tasks.

Instruction-following failures are the most prominent errors leading to many aborted games. Namely, the response tag was incorrect or missing; multiple answers were provided within a single turn; answers were given in Chinese instead of English (Figure 1). A small number of hallucinations was also observed, where the player appeared to "fall out" of the game process (Figure 2). These are precisely the issues our experiments aim to address.

#	A	Game Master	B
1		[P1] (GM) WAITER: What is your choice for the main dish?	
2	[P1] (GM) 顾客: 我想要一份牛排, 七分熟, 配烤蔬菜和土豆泥。		
3		[GM] (P2) What is the next request?	
4		[GM] (P2) Do you also want a salad?	
5		[P1] (GM) WAITER: Do you also want a salad?	
6	[P1] (GM) 顾客: 是的, 来一份凯撒沙拉吧, 谢谢。		

Figure 1: Interaction Transcript for game *private-shared*, experiment *restaurant*, episode 4, Baseline [Excerpt]

3.2 The Pre-Game Planning Agent

The goal of introducing a Pre-Game Planning Agent is to extend the decision-making process compared to the immediate-response setting required by **clembench**. To this end, we introduce an additional reasoning step: the model is provided with the full game history and prompted to explicitly reason what the possible next step could be. It

#	A	Game Master	B
1	[P1] (GM) GO: east		
2		[GM] (P2) GO: east	
3		[GM] (P2) You have made a step and entered Game room. Currently available directions: west. What is your next instruction?	
4		[GM] (GM) Continue	
5		[GM] (GM) {"old": "Bedroom", "new": "Game room"}	
6		[P1] (GM) You have made a step and entered Game room. Currently available directions: west. What is your next instruction?	
7	[P1] (GM) GO: west		
8		[GM] (GM) You have made a step and entered Bedroom. Currently available directions: south, east, west, north. What is your next instruction?	
9		[GM] (GM) abort game	

Figure 2: Interaction Transcript for game *textmapworld*, experiment *medium_cycle*, episode 18, Baseline [Excerpt]

then outputs the single best next action, followed by an "ACT:" tag. The part of answer after the tag is then extracted and passed to the Game Master (Figure 3).

Here, *planning* is understood in a simplified sense:

Planning

Given a goal G , decide on a sequence of actions $(a_0, a_1, \dots, a_n)$ that leads to a state passing the goal test $g(\cdot)$ (Russell et al., 1995).

In case of **clembench**, $g(\cdot)$ maps a game's final state to {success, loss, abort}, and passing the goal test can be interpreted as $g(\text{final state}) = \text{success}$.

We expected that this additional reasoning step will reduce the number of hallucinations, as the model is allowed to reprocess information from the environment with a "fresh glance". To improve efficiency, the maximum number of tokens produced in a single model call is limited to 300.

The results are summarized in Table 2.

The Pre-Game Planning agent achieves the clemscore of **8.22**. The results vary substantially across game clusters, with performance degrading in most categories compared to the baseline.

The *word-based* game cluster shows the weakest result overall. *Wordle* drops to 0% played, mostly because the model was outputting only the ACT: tag and then empty line. *Guesswhat* still fails completely. *Taboo* maintains a 60% completion rate but achieves a very low quality score of 11.11 with high variance, indicating inconsistent performance. *Codenames* completes only

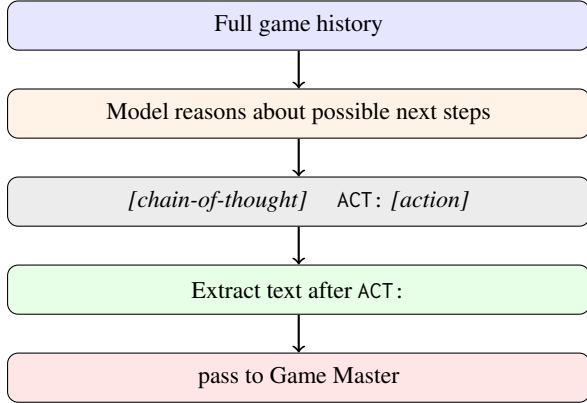


Figure 3: Pre-Game Planning Agent pipeline.

Game	clemscore	% Played	Qlty Score
adventuregame		0.0 ↓	NaN ↓
codenames		7.69 ↓	50.0 (70.71) ↓
guesswhat		0.0	NaN
imagegame		0.0	NaN
matchit		0.0 ↓	NaN ↓
matchit_ascii		100.0	0.0 ↓
multimodal_mapworld	8.22	100.0 ↑	59.01 (20.31)
multimodal_referencegame		0.0 ↓	NaN ↓
privateshared		88.0 ↓	NaN ↓
referencegame		0.0 ↓	NaN ↓
taboo		60.0 ↓	11.11 (33.33) ↓
textmapworld		0.0	NaN
wordle		0.0 ↓	NaN ↓

Table 2: Metrics for the Pre-Game Planning Agent (Qwen3-VL-30B-A3B-Instruct-FP8). Colored arrows indicate change relative to baseline.

#	A	Game Master	B
1	[P1 GM]	Let's do the following task. <...> [Initial prompt omitted for brevity.]	
2	[P1 GM]	Comand: Put a J in first row second column	
3		[GM GM]	Invalid instruction format

Figure 4: An aborted round of *imagegame* demonstrating bad format compliance [Simulated interaction, excerpt]

7.69% of episodes, though the quality score of 50.0 should be interpreted cautiously given the high standard deviation (70.71) and very low sample size.

Multimodal_mapworld is the strongest result of the Pre-Game Planning Agent, achieving a **+16%** played while maintaining the more or less same quality score. In contrast, both *adventuregame* and *textmapworld* fail completely, and this does not allow us to state that explicit reasoning may benefit exploration tasks.

Finally, performance in *image-based games* cluster also falls significantly. *Referencegame* and *multimodal_referencegame* drop from 100 to 0% played. *Matchit* also fails completely, while *matchit_ascii* achieves full completion but a quality score of 0.0, indicating that episodes are played but without any correct matches. *Imagegame* remains at 0% as in the baseline.

Given the model’s already limited instruction-following in the baseline, introducing an additional reasoning step may have compounded existing format compliance issues. Nevertheless, this experiment was a necessary proof of concept.

3.3 The Format-Recovery Agent

Since **clmbench** employs strict rules for response formatting, many games are aborted due to incorrectly formatted answers, even when the underlying responses are semantically and strategically correct. Consider the example in Figure 4: a simple typo ("Comand" instead of "Command") leads to the game being aborted immediately. Moreover, only a small subset of the evaluated games allows for reprompting the player after a formatting error (namely, *wordle*, *privateshared*, *mapworld* and *codenames*). With our proposed addition, all games are treated uniformly in that regard.

Therefore, another natural extension to the basic LLM setup is to introduce a validation

loop in which the player’s response is checked against the specified output format. For example, a certain mechanism (an LLM judge or a regular-expression-based parser) could extract the required tags from the Game Master’s prompt and verify whether the Player’s intended response contains them (Figure 3).

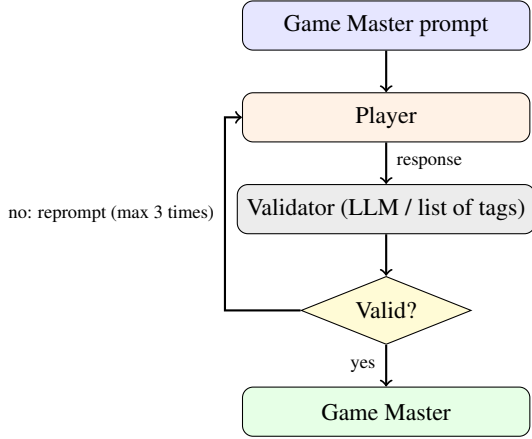


Figure 5: Format-Recovery Agent: the player’s response is validated before being passed to the Game Master. On failure, a reprompt is issued and the loop repeats.

This add-on is mainly aimed to improve the % played score component. Below we compare the % played for a Format-Recovery Agent based on a fixed game tags set and an LLM judge (deepseek-v3.2) with the baseline (Table 3). Full results can be found in the Appendix (Tables 8, 9) .

Game	Baseline (%)	Exp1 (%)	Exp2 (%)
adventuregame	79.17	20.83 ↓	20.83 ↓
codenames	38.46	30.77 ↓	34.62 ↓
guesswhat	0.0	20.83 ↑	33.33 ↑
imagegame	0.0	0.0	0.0
matchit	95.83	0.0 ↓	0.0 ↓
matchit_ascii	100.0	100.0	100.0
mm_mapworld	84.0	28.0 ↓	28.0 ↓
mm_referencegame	100.0	100	100
privateshared	100.0	84.21 ↓	100.0
referencegame	100.0	100.0	100.0
taboo	100.0	100.0	93.33 ↓
textmapworld	0.0	0.0	0.0
wordle	70.0	30.0 ↓	40.0 ↓
clemscore	26.61	16.54 ↓	17.9 ↓

Table 3: Comparison of % Played and clemscores across baseline and experiments with format checking. Exp1 indicates an agent with a predefined set of tags. Exp2 indicates an agent with an LLM judge extracting game tags. Colored arrows indicate change relative to baseline.

Contrary to expectations, the format checking loop does not improve overall performance — the

clemscore drops from 26.61 to 16.54 (Exp1) and 17.9 (Exp2). The only clear improvement is *guesswhat*, which increases from 0% to **20.83%** and **33.33%** respectively, suggesting that reprompting helps in games where a single tag violation led to the game being aborted immediately.

However, this is not true for all games. The most likely explanation is that the checker was implemented with an *if-any* condition — verifying the presence of at least one valid tag, which is insufficient for games requiring multiple tags simultaneously. *Matchit* is even more complex: it requires different tags at different stages of the game (DESCRIPTION: early on, DECISION: near the end), so neither a single-tag check nor a static per-game config can capture the turn-dependent format. Additionally, the checker does not handle the case where the model produces multiple tags in sequence within a single response, which is also grounds for abort in several games.

The core problem is that the agent needs to "know" the expected format *per game per turn*, which cannot be parsed from the game prompt alone, since some tags appear close to the game’s end (consequently, errors like Figure 6 happen). A per-game configuration specifying which tags are required and whether all or only one is expected per turn would address the simpler cases, but turn-dependent formats such as *matchit* would require a more dynamic solution.

Notably, unlike the Pre-Game Planning Agent, this approach does not disrupt *referencegame* and *multimodal_referencegame*, both of which maintain 100% played. This suggests the format checker is compatible with single-tag, single-turn response formats.

The further cluster analysis is not provided here, since formatting rules are defined per game rather than per cluster.

3.4 The Reflexion Agent

This experiment is an attempt to adapt the Reflexion approach (Shinn et al., 2023) — a paradigm for "verbal" reinforcement where agents reflect on task feedback and store these self-reflections in their memory to ensure better decision making in the future. No weight updates occur.

The components of Reflexion are mapped to **clembench** components as follows:

```

# | A                                     Game Master                                     B
1 | [GM|P2] What class do you prefer?
2 | [P1|GM] TRAVEL-AGENT: What class do you prefer?
3 | [P1|internal] ['TRAVEL-AGENT:']
   | [P1|internal] Please provide the following
   | details so I can assist you with
   | your travel arrangements:
   | 1. **Place of Departure**: Where
   | are you starting your journey from?
   | 2. **Preferred Transportation**:
   | Do you prefer flying, train, car, or
   | another mode of transport?
   | 3. **Trip Start Date**: When would
   | you like your trip to begin?
   | 4. **Preferred Class**: Do you
   | prefer Economy, Business, or First
   | Class?
   | Let me know, and I'll help you plan
   | your trip!
   | [P1|internal] [attempt 2 { valid}]
   | TRAVEL-AGENT: Please inform your
   | place of departure.
   | TRAVEL-AGENT: What kind of
   | transportation do you want to use?
5 | TRAVEL-AGENT: When does your trip
   | begin?
   | TRAVEL-AGENT: What class do you
   | prefer?
   | [P1|GM] TRAVEL-AGENT: Please inform your place of
   | departure.
   | TRAVEL-AGENT: What kind of transportation do you
   | want to use?
   | TRAVEL-AGENT: When does your trip begin?
   | TRAVEL-AGENT: What class do you prefer?
6 |
7 | [GM|P2] What is the next request?

```

Figure 6: Interaction Transcript for game *private-shared*, experiment *travel-booking*, episode 3, The Format-Recovery Agent. Due to the wrong tag extraction, an invalid answer is passed on. The game is not aborted, but the quality score decreases. [Excerpt]

Reflexion

An Actor, denoted as $M_a \rightarrow$ the **Player** (generates text and actions);
 An Evaluator model, represented by $M_e \rightarrow$ the **Game Master** (scores the outputs produced by M_a)
 A Self-Reflection model, denoted as $M_{sr} \rightarrow$ **another LM** (generates verbal reinforcement cues to assist the Actor in self-improvement)

Note that the Game Master is chosen as the Evaluator rather than the Game Scorer, because in **clmbench** scoring occurs after the game run finishes — players do not have access to scores during play. The Game Master only logs raw data (turn events, outcomes, request counts). For now, feedback is based on the Game Master’s messages to each player. For example:

- Round 1: M_a produces the answer
- Round 1: M_e gives feedback (e.g., "Your answer should contain the following tags: <...>")
- Round 1: M_{sr} reflects on the answer given and the feedback and outputs *reflexion*

- Round 2: M_a receives the next Game Master prompt together with the *reflexion*.

Another key component is memory. For now, only the last 3 reflections are stored and passed on to the Player. Each Player maintains its own reflection memory. To avoid having the same model both play and reflect on its own outputs, we use deepseek-v3 as M_{sr} . To improve efficiency, the maximum number of tokens produced in a single model call is limited to 300.

Game	clmscore	% Played	Qlty Score
adventuregame		12.5 ↓	33.33 (57.74) ↑
codenames		26.92 ↓	42.86 (53.45) ↑
guesswhat		8.33 ↑	0.0
imagegame		0.0	NaN
matchit		0.0 ↓	NaN ↓
matchit_ascii		100.0	0.0 ↓
multimodal_mapworld	7.69	32.0 ↓	50.09 (8.5) ↓
multimodal_referencegame		0.0 ↓	NaN ↓
privateshared		96.0 ↓	NaN
referencegame		0.0 ↓	NaN ↓
taboo		100.0	23.33 (41.69) ↓
textmapworld		0.0	NaN
wordle		40.0 ↓	18.75 (37.2) ↓

Table 4: Metrics for the Reflexion Agent Qwen3-VL-30B-A3B-Instruct-FP8. Colored arrows indicate change relative to baseline.

Results (Table 4) show that the Reflexion is the worst-performing configuration overall (**-18.92** clmscore change). This decrease is driven primarily by a collapse in *% played*: across most games, the model increasingly fails to produce valid outputs, likely because the added Reflexion context disrupts format adherence. Where the Reflexion agent does complete episodes, quality sometimes improves (e.g., *adventuregame*, *codenames*), but this should be interpreted cautiously — completion rates are so low that absolute performance remains worse despite the apparent gains. The only notable exception is *guesswhat*, which recorded 0.0% played in the baseline and some other tested configurations, yet reached 8.33% under Reflexion; however, the quality score remains 0.0.

Cluster-level analysis reveals that Reflexion was most damaging for image-based games and least damaging for word-based games, probably because "verbal" feedback is better suited to tasks where language itself is the primary medium. Exploration games present a paradox: *adventuregame* achieves the largest quality gain in the entire evaluation (**+28** pts), yet the completion rate drops so sharply that this improvement is rarely realized. We are inclined to state that Reflexion is highly sensitive to game modality.

3.5 The Upfront-Strategy Agent

The last configuration evaluates whether injecting explicit, human-authored game strategies directly into the player’s prompt improves performance. Rather than relying on the model to discover effective strategies through interaction or verbal reinforcement, the strategy is delivered to the player as an internal message within the conversation history (Figure 7). It remains unchanged throughout the game and is repeated every turn. The strategy messages can be found in Appendix 10.

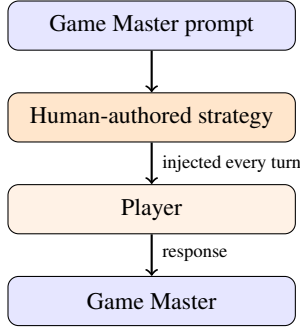


Figure 7: Upfront-Strategy Agent: a fixed, human-authored strategy is injected into the player’s conversation history at every turn.

This add-on is intended to increase the rate of successfully completed turns, as the Agent does not need to recover the same knowledge from context or game flow. This makes it cheaper than Reflexion, but rigidity is the tradeoff here: the injected strategy is fixed, cannot adapt to unexpected game states (which is vital for not-mundane games where the flow is changing, e.g., *matchit*), and is only as good as the annotator who wrote it.

Game	clemscore	% Played	Qlty Score
adventuregame		29.17 ↓	33.33 (33.33) ↓
codenames		30.77 ↓	37.5 (51.75) ↑
guesswhat		0.0	NaN
imagegame		0.0	NaN
matchit		4.17 ↓	100.0 ↑
matchit_ascii		100.0	37.5 (49.45) ↓
multimodal_mapworld	28.41	84.0	58.23 (15.39) ↓
multimodal_referencegame		100.0	86.36 (35.13) ↑
privateshared		100.0	NaN
referencegame		100.0	80.0 (40.82) ↑
taboo		100.0	33.33 (44.99) ↑
textmapworld		0.0	NaN
wordle		45.0 ↓	13.33 (33.17) ↑

Table 5: Metrics for the Upfront-Strategy Agent Qwen3-VL-30B-A3B-Instruct-FP8. Colored arrows indicate change relative to baseline.

The Upfront-Strategy Agent (Table 5) is the first case where an add-on shows an improvement (+1.80 clemscore). For some *word-based games*, quality improves while % played holds steady

(e.g., taboo + 6.7, codenames + 7.5), but given the high std, these differences should be interpreted cautiously; whereas for wordle, % played decreases, making the Quality score improvement not meaningful (although see Figure 8).

There are also improvements in *image-based* games, namely, in two versions of *referencegame* (+8; +9). However, they are canceled out by *imagegame*, which still stands by zero, and *matchit*, where 95.83% played drops to 4.17%, with the single surviving episode scoring 100.

As for *exploration* games, the strategy injection does not seem to influence them at all. Textmapworld still has zero episodes played, adventuregame’s % played collapses from 79 to 29, multimodal_mapworld stagnates.

The strategy injection improves decision quality but at a cost to format compliance, especially in structured multi-turn games (*matchit*, *adventuregame*, *wordle*). The clemscore gain is small because the quality gains are partially canceled out by more aborts. The agent is thinking better, but formatting worse.

4 From Targeted Modifications to Full Agent Frameworks

The aforementioned configurations targeted a specific weakness through a lightweight, interpretable modification. As an opposite end of this architectural spectrum, it is of interest to offload the agent behaviour to an established, general-purpose framework. This allows us to ask whether off-the-shelf agent infrastructure can match or outperform hand-crafted, targeted interventions.

For this purpose, we used [LangChain](#), a Python library for building custom agents. LangChain agents are built on the ReAct paradigm (Yao et al., 2022): at each step, the model receives the full conversation history and decides either to call a tool or produce a final response. If a tool is called, its result is appended to the conversation and the model is invoked again; this repeats until a plain-text response is returned with no tool call. State across turns is managed by a checkpoint, which stores the full message history keyed by a thread id, giving the agent persistent within-episode memory without explicit context management.

Some parallels with **clembench** are visible (e.g., the ClemAgent history vs. the LangChain checkpoint). However, a LangChain agent without

tools is not a meaningful point of comparison against the **clmbench** baseline: when no tools are defined, the Reason – Act loop degenerates to a single step, and the agent behaves identically to a plain chat model. The framework adds no functional capability beyond what the baseline already provides.

For this reason, we report only the results of the tool-augmented configuration (the Memory-Tool Agent), which is a LangChain setup that introduces behaviour genuinely beyond the baseline. We acknowledge, however, that this experiment is not directly comparable to the other configurations described in this paper: unlike the targeted add-ons, it relies on a different execution framework, a different model interface, and tool-calling infrastructure, making isolated attribution of any performance difference difficult.

The Memory-Tool Agent is built on the observation that LLMs have no persistent perception, they cannot sense the current game state directly (e.g., the rooms in *textmapworld*). This makes them prone to acting on old or hallucinated information, particularly in longer games where early game state is pushed far back in the conversation history.

To address this, the agent is equipped with two tools: `observe_game()`, which allows the agent to record a notable observation in its own words, and `get_game_observations()`, which retrieves the full list of recorded observations. The agent follows the following turn protocol: on the first turn it receives the game rules and responds directly; on every subsequent turn it is required to call both tools — first recording a new observation, then reviewing the accumulated record before producing its response.

Game	clmscore	% Played	Qlty Score
adventuregame		70.83	17.65 (35.59)
codenames		46.15	16.67 (38.92)
guesswhat		95.83	27.54 (32.8)
imagegame		41.67	89.6 (11.98)
matchit		100.0	54.17 (50.9)
matchit_ascii		100.0	87.5 (33.78)
mm_mapworld	32.31	44.0	63.95 (10.44)
multimodal_referencegame		100.0	27.27 (45.58)
privateshared		88.0	NaN
referencegame		92.0	73.91 (44.9)
taboo		100.0	20.0 (36.84)
textmapworld		84.0	62.76 (13.37)
wordle		75.0	2.0 (7.75)

Table 6: Metrics for the Memory-Tool Agent (Qwen3-VL-30B-A3B-Instruct-FP8).

The Memory-Tool Agent achieves a clmscore

of **32.31** (Table 6), the highest of all tested configurations, though the comparability caveat from the previous section applies.

The most striking result is the activation of three games that recorded 0% played in the baseline: *guesswhat* (0.0 → 95.83), *imagegame* (0.0 → 41.67), and *textmapworld* (0.0 → 84.0). This supports the grounding hypothesis: explicit observation recording appears to help the agent maintain coherent game state in games where pure context reconstruction consistently failed.

Also, the result raises a broader question: rather than engineering targeted, hand-crafted modifications to a base player, it may be more effective to delegate agent behaviour to a pre-coded library then to compare incremental modifications to a base LLM player.

5 Conclusion

This work investigated whether lightweight modifications to a base LLM player can improve performance on **clmbench** dialogue games, and whether a general-purpose agent framework offers an advantage over such modifications.

The answer to the first question is negative. It turned out that injecting the strategy information is the only configuration that helped raise the performance (+1.80). All other added structures increased complexity and led to more aborted games. This suggests that architectural enhancements do not straightforwardly transfer to dialogue gameplay. The dominant reason for failure across all configurations is format compliance.

On the contrary, The LangChain Memory-Tool Agent achieves the highest clmscore (32.31), and its success on previously broken games suggests that tool-mediated grounding is a more robust approach than prompt-based augmentation. Whether this advantage can be augmented across models and benchmarks remains an open question.

Limitations

Our study has several limitations. Firstly, all experiments were conducted within a single benchmark, on a single model and for one language only, which generally does not allow for any generalization of the findings. The results may differ substantially for alternative benchmarks such as TextArena (Guertler et al., 2025).

Secondly, the Upfront-Strategy Agent’s strategies are static and model-specific, contradicting

the goal of a general-purpose agent: they cannot adapt to model-specific failures.

Finally, some inconsistencies in the benchmark make results harder to interpret uniformly. For example, *privateshared* does not abort on incorrect or missing tags, unlike most other games (the game is considered lost but played), which means its % played metric is not directly comparable to the rest of the benchmark.

Ethical Considerations

We do not see any critical aspects of our work with respect to ethical considerations. All experiments were conducted using publicly available models and benchmarks.

References

- Anthropic. 2024. [Introducing computer use, a new Claude 3.5 Sonnet, and Claude 3.5 Haiku](#). Accessed: April 4, 2026.
- Kranti Chalamalasetti, Jana Götze, Sherzod Hakimov, Brielen Madureira, Philipp Sadler, and David Schlangen. 2023. *clmbench*: Using game play to evaluate chat-optimized language models as conversational agents. In *Proceedings of the 2023 conference on empirical methods in natural language processing*, pages 11174–11219.
- Leon Guertler, Bobby Cheng, Simon Yu, Bo Liu, Leshem Choshen, and Cheston Tan. 2025. Textarena. *arXiv preprint arXiv:2504.11442*.
- Sherzod Hakimov, Yerkezhan Abdullayeva, Kushal Koshti, Antonia Schmidt, Yan Weiser, Anne Beyer, and David Schlangen. 2025. [Using game play to investigate multimodal and conversational grounding in large multimodal models](#). In *Proceedings of the 31st International Conference on Computational Linguistics*, pages 5686–5718, Abu Dhabi, UAE. Association for Computational Linguistics.
- Sherzod Hakimov, Roland Bernard, Tim Leiber, Karl Osswald, Kristina Richert, Ruilin Yang, Raffaella Bernardi, and David Schlangen. 2026. [The price of thought: A multilingual analysis of reasoning, performance, and cost of negotiation in large language models](#). In *Findings of the Association for Computational Linguistics: EACL 2026*, pages 529–570, Rabat, Morocco. Association for Computational Linguistics.
- Nicola Horst, Davide Mazzaccara, Antonia Schmidt, Michael Sullivan, Filippo Momentè, Luca Franceschetti, Philipp Sadler, Sherzod Hakimov, Alberto Testoni, Raffaella Bernardi, Raquel Fernández, Alexander Koller, Oliver Lemon, David Schlangen, Mario Giulianelli, and Alessandro Suglia. 2025. [Playpen: An environment for exploring learning from dialogue game feedback](#). In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 29854–29891, Suzhou, China. Association for Computational Linguistics.
- Nikolai Illykh, Sina Zarriß, and David Schlangen. 2019. Meetup! a corpus of joint activity dialogues in a visual environment. *arXiv preprint arXiv:1907.05084*.
- Chris Manning. 2025. Keynote: Linguistic agents that learn from interactions. In *Proceedings of the 1st Workshop for Research on Agent Language Models (REALM 2025)*, Vienna, Austria. Association for Computational Linguistics.
- Grégoire Mialon, Roberto Dessì, Maria Lomeli, Christoforos Nalmpantis, Ram Pasunuru, Roberta Raileanu, Baptiste Rozière, Timo Schick, Jane Dwivedi-Yu, Asli Celikyilmaz, and 1 others. 2023. Augmented language models: a survey. *arXiv preprint arXiv:2302.07842*.
- Aske Plaat, Max Van Duijn, Niki Van Stein, Mike Preuss, Peter Van der Putten, and Kees Joost Batenburg. 2025. [Agentic large language models, a survey](#). *Journal of Artificial Intelligence Research*, 84.
- Shanghaoran Quan, Jiaxi Yang, Bowen Yu, Bo Zheng, Dayiheng Liu, An Yang, Xuancheng Ren, Bofei Gao, Yibo Miao, Yunlong Feng, Zekun Wang, Jian Yang, Zeyu Cui, Yang Fan, Yichang Zhang, Binyuan Hui, and Junyang Lin. 2025. [Codeelo: Benchmarking competition-level code generation of llms with human-comparable elo ratings](#). *Preprint*, arXiv:2501.01257.
- Stuart Russell, Peter Norvig, and Artificial Intelligence. 1995. A modern approach. *Artificial Intelligence*. Prentice-Hall, Englewood Cliffs, 25(27):79–80.
- David Schlangen. 2023. [Dialogue games for benchmarking language understanding: Motivation, taxonomy, strategy](#). *Preprint*, arXiv:2304.07007.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in neural information processing systems*, 36:8634–8652.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*.
- Bingchen Zhao, Jenny Zhang, Chenxi Whitehouse, Minqi Jiang, Michael Shvartsman, Abhishek Charnalia, Despoina Magka, Tatiana Shavrina, Derek Dunfield, Oisín Mac Aodha, and Yoram Bachrach. 2026. [Apres: An agentic paper revision and evaluation system](#). *Preprint*, arXiv:2603.03142.

Game	Description	Players	Type
adventuregame	The player is given descriptions of rooms and their contents, along with a task, and interacts with the game world by giving action commands.	1	text-based
codenames	Simulates the well-known Codenames game in one team only. The player gives one-word clues to help their teammate identify assigned words from a shared grid, while avoiding the opposing team's words and the hidden kill word.	2	text-based
imagegame	One player describes how to fill cells in a grid using referring expressions, while the other incrementally updates an initially empty grid. The process continues until the target grid is reconstructed or the instruction giver signals completion.	2	text-based (ASCII)
matchit	Two players must agree on whether the image each of them receives as input is the same or not.	2	image-based
matchit_ascii	Two players must agree on whether the ASCII grid each of them receives as input is the same or not.	2	text-based (ASCII)
multimodal_referencegame	The first player generates a referring expression that uniquely identifies a target image among two distractors. The second player selects the correct image from the shuffled set.	2	image-based
mm_mapworld	The player explores a network of connected rooms using visual cues. Based on the MapWorld environment (Ilinykh et al., 2019).	1	image-based
privateshared	A slot-filling conversation game. The questioner asks questions to fill unknown slots; the answerer privately knows all slot values. The goal is to correctly complete all slots by the end of the interaction.	2	text-based
referencegame	The first player generates a referring expression that uniquely identifies a target grid among two distractors. The second player selects the correct grid from the shuffled set.	2	text-based
taboo	The describer must convey a target word to a guesser without using the word itself or a set of related forbidden words. The guesser has three attempts.	2	text-based
textmapworld	The player explores a network of connected rooms by issuing navigation commands. Based on the MapWorld environment (Ilinykh et al., 2019).	1	text-based
wordle	One player iteratively guesses a target word; the other provides letter-level feedback on each guess.	2	text-based

Table 7: Overview of clembench games used in this study.

A Full list of used clembench games

For the full list of games used, see Table 7.

B Example: A Successful Taboo Game

"The following transcript shows a successful episode of *taboo* played by the Memory-Tool Agent, in which the target word *redirect* is guessed in two turns.

#	A	Game Master	B
1		[P1]GM] You are playing a collaborative word guessing game in which you have to describe a target word for another player to guess. Rules: (a) You have to reply in the form: CLUE: <some text>. Guesses from the other player will start with GUESS. (b) You cannot use the target word itself, parts or morphological variants of it in your description. (c) In addition, the same rules apply for related words which are provided below. End conditions: (i) If you use the target word or a related word in your description, then you lose. (ii) If the other player can guess the target word in 3 tries, you both win. Let us start. This is the target word that you need to describe and that the other player needs to guess: redirect Related words are: - reroute - forward - divert Important: You are under time pressure, give short descriptions that are to the point!	

2	[P1]GM] CLUE: Send something in a different direction	
3		[GM]GM] continue
4		[GM]GM] send something in a different direction
5		[GM]P2] You are playing a collaborative word guessing game in which you have to guess a target word that another player describes to you. You can make one guess at each trial. You win when you guess the target word. You lose when you cannot guess it in 3 tries. After each trial you will get a new hint from the other player which starts with CLUE. Make your guesses by just saying the word using the following form: GUESS: <a word> Let us start. CLUE: Send something in a different direction
6		[GM]P2] GUESS: arrow
7		[GM]GM] continue
8		[GM]GM] arrow
9		[P1]GM] GUESS: arrow
10	[P1]GM] CLUE: Change the path of something	
11		[GM]GM] continue
12		[GM]GM] change the path of something
13		[GM]P2] CLUE: Change the path of something
14		[GM]P2] GUESS: redirect
15		[GM]GM] continue
16		[GM]GM] redirect
17		[GM]GM] end game

C Full Results for theFormat-Recovery Agent

Game	clemscore	% Played	Qlty Score
adventuregame		20.83	86.67 (18.26)
codenames		30.77	50.0 (53.45)
guesswhat		20.83	0.0
imagegame		0.0	NaN
matchit		0.0	NaN
matchit_ascii		100.0	0.0
multimodal_mapworld	16.54	28.0	49.68 (12.94)
multimodal_referencegame		100.0	68.18 (47.67)
privateshared		84.21	0.0
referencegame		100.0	72.0 (45.83)
taboo		100.0	23.33 (41.69)
textmapworld		0.0	NaN
wordle		30.0	0.0

Table 8: Metrics for the Format-Recovery Agent (Qwen3-VL-30B-A3B-Instruct-FP8), set of tags predefined.

Game	clemscore	% Played	Qlty Score
adventuregame		20.83	53.33 (50.55)
codenames		34.62	44.44 (52.7)
guesswhat		33.33	0.0
imagegame		0.0	NaN
matchit		0.0	NaN
matchit_ascii		100.0	0.0
multimodal_mapworld	17.9	28.0	50.79 (10.81)
multimodal_referencegame		100.0	77.27 (42.89)
privateshared		100.0	NaN
referencegame		100.0	64.0 (48.99)
taboo		93.33	28.57 (46.88)
textmapworld		0.0	NaN
wordle		40.0	3.75 (10.61)

Table 9: Metrics for the the Format-Recovery Agent (Qwen3-VL-30B-A3B-Instruct-FP8) with an LLM judge.

D Upfront-Strategy Agent Prompt Injection

For the game strategies, see Table 10.

E A game successfully played by the Upfront-Strategy Agent

See Figure 8.

#	Player	Game Master	#	Player	Game Master
1	(human) Start with high-frequency letters. Use each guess to eliminate as many letters as possible. Never repeat a letter already marked red! Never repeat a guess already chosen wrong.		1	(human) explanation: I am starting with a common five-letter word that includes frequently used vowels and consonants to gather initial information about the target word.	
2	(human) guess: crane	(human) guess.feedback: a<red> d<red> i<green> e<red> u<red>	2	(human) Provide your response only in this format. explanation: details guess: word	
3	(human) explanation: I am starting with "crane" because it contains common vowels (a, e) and high-frequency consonants (c, r, n). <...>		3	(human) explanation: The feedback shows that only the letter 'i' is correct and in the correct position (6th position). The other letters (a, d, e, u) are incorrect. I will exclude these letters and focus on finding a word with 'i' in the 6th position, using other common vowels and consonants.	
4	(human) Start with high-frequency letters		4	(human) Error: The guess should be a single word and should only contain letters.	
5	(human) explanation: All letters in "crane" are incorrect, so I will avoid vowels (a, e) and high-frequency consonants (c, r, n). <...>				
6	(human) guess: blind	(human) guess.feedback: b<green> i<green> i<green> n<red> d<red>			
7	(human) Provide your response only in this format. explanation: details guess: word				
8	(human) Start with high-frequency letters				
9	(human) explanation: The feedback shows that <...>				
10	(human) guess: blind	(human) guess.feedback: b<green> i<green> i<green> n<red> d<red>			
11	(human) Provide your response only in this format. explanation: details guess: word				
12	(human) Start with high-frequency letters				
13	(human) explanation: The feedback shows that <...>				
14	(human) guess: blimp	(human) guess.feedback: b<green> i<green> i<green> n<red> d<red>			
15	(human) Provide your response only in this format. explanation: details guess: word				
16	(human) Start with high-frequency letters				
17	(human) explanation: The feedback shows that <...>				
18	(human) guess: blimp	(human) guess.feedback: b<green> i<green> i<green> n<red> d<red>			
19	(human) Provide your response only in this format. explanation: details guess: word				
20	(human) Start with high-frequency letters				
21	(human) explanation: The feedback shows that <...>				
22	(human) guess: blimp	(human) guess.feedback: b<green> i<green> i<green> n<red> d<red>			
23	(human) Provide your response only in this format. explanation: details guess: word				
24	(human) Start with high-frequency letters				
25	(human) explanation: The feedback shows that <...>				
26	(human) guess: blimp	(human) guess.feedback: b<green> i<green> i<green> n<red> d<red>			
27	(human) Provide your response only in this format. explanation: details guess: word				
28	(human) Start with high-frequency letters				
29	(human) explanation: The feedback shows that <...>				
30	(human) guess: blimp	(human) guess.feedback: b<green> i<green> i<green> n<red> d<red>			
31	(human) Provide your response only in this format. explanation: details guess: word				
32	(human) Start with high-frequency letters				
33	(human) explanation: The feedback shows that <...>				
34	(human) guess: blimp	(human) guess.feedback: b<green> i<green> i<green> n<red> d<red>			
35	(human) Provide your response only in this format. explanation: details guess: word				
36	(human) Start with high-frequency letters				
37	(human) explanation: The feedback shows that <...>				
38	(human) guess: blimp	(human) guess.feedback: b<green> i<green> i<green> n<red> d<red>			
39	(human) Provide your response only in this format. explanation: details guess: word				
40	(human) Start with high-frequency letters				
41	(human) explanation: The feedback shows that <...>				
42	(human) guess: blimp	(human) guess.feedback: b<green> i<green> i<green> n<red> d<red>			
43	(human) Provide your response only in this format. explanation: details guess: word				
44	(human) Start with high-frequency letters				
45	(human) explanation: The feedback shows that <...>				
46	(human) guess: blimp	(human) guess.feedback: b<green> i<green> i<green> n<red> d<red>			
47	(human) Provide your response only in this format. explanation: details guess: word				
48	(human) Start with high-frequency letters				
49	(human) explanation: The feedback shows that <...>				
50	(human) guess: blimp	(human) guess.feedback: b<green> i<green> i<green> n<red> d<red>			
51	(human) Provide your response only in this format. explanation: details guess: word				
52	(human) Start with high-frequency letters				
53	(human) explanation: The feedback shows that <...>				
54	(human) guess: blimp	(human) guess.feedback: b<green> i<green> i<green> n<red> d<red>			
55	(human) Provide your response only in this format. explanation: details guess: word				
56	(human) Start with high-frequency letters				
57	(human) explanation: The feedback shows that <...>				
58	(human) guess: blimp	(human) guess.feedback: b<green> i<green> i<green> n<red> d<red>			
59	(human) Provide your response only in this format. explanation: details guess: word				
60	(human) Start with high-frequency letters				
61	(human) explanation: The feedback shows that <...>				
62	(human) guess: blimp	(human) guess.feedback: b<green> i<green> i<green> n<red> d<red>			
63	(human) Provide your response only in this format. explanation: details guess: word				
64	(human) Start with high-frequency letters				
65	(human) explanation: The feedback shows that <...>				
66	(human) guess: blimp	(human) guess.feedback: b<green> i<green> i<green> n<red> d<red>			
67	(human) Provide your response only in this format. explanation: details guess: word				
68	(human) Start with high-frequency letters				
69	(human) explanation: The feedback shows that <...>				
70	(human) guess: blimp	(human) guess.feedback: b<green> i<green> i<green> n<red> d<red>			
71	(human) Provide your response only in this format. explanation: details guess: word				
72	(human) Start with high-frequency letters				
73	(human) explanation: The feedback shows that <...>				
74	(human) guess: blimp	(human) guess.feedback: b<green> i<green> i<green> n<red> d<red>			
75	(human) Provide your response only in this format. explanation: details guess: word				
76	(human) Start with high-frequency letters				
77	(human) explanation: The feedback shows that <...>				
78	(human) guess: blimp	(human) guess.feedback: b<green> i<green> i<green> n<red> d<red>			
79	(human) Provide your response only in this format. explanation: details guess: word				
80	(human) Start with high-frequency letters				
81	(human) explanation: The feedback shows that <...>				
82	(human) guess: blimp	(human) guess.feedback: b<green> i<green> i<green> n<red> d<red>			
83	(human) Provide your response only in this format. explanation: details guess: word				
84	(human) Start with high-frequency letters				
85	(human) explanation: The feedback shows that <...>				
86	(human) guess: blimp	(human) guess.feedback: b<green> i<green> i<green> n<red> d<red>			
87	(human) Provide your response only in this format. explanation: details guess: word				
88	(human) Start with high-frequency letters				
89	(human) explanation: The feedback shows that <...>				
90	(human) guess: blimp	(human) guess.feedback: b<green> i<green> i<green> n<red> d<red>			
91	(human) Provide your response only in this format. explanation: details guess: word				
92	(human) Start with high-frequency letters				
93	(human) explanation: The feedback shows that <...>				
94	(human) guess: blimp	(human) guess.feedback: b<green> i<green> i<green> n<red> d<red>			
95	(human) Provide your response only in this format. explanation: details guess: word				
96	(human) Start with high-frequency letters				
97	(human) explanation: The feedback shows that <...>				
98	(human) guess: blimp	(human) guess.feedback: b<green> i<green> i<green> n<red> d<red>			
99	(human) Provide your response only in this format. explanation: details guess: word				
100	(human) Start with high-frequency letters				

Figure 8: A comparison of two wordle episodes: The Upfront-Strategy Agent on the left and the Baseline Agent on the right [Excerpt].The target word is *blimp*. The game on the left is won after 5 turns. The game on the right is aborted, because a *guess:* tag was not followed by a word.

Game	Injected Strategy
adventuregame	Explore all directions before backtracking. Remember item locations. Read room descriptions carefully. If walking in circles, write > done.
codenames	Find semantic clusters among your team words. Prioritize clues covering multiple words at once while staying far from opponent and assassin words. Do not repeat words already guessed.
guesswhat	Start with broad category questions to eliminate large portions of the space. Progressively narrow down with specific attributes.
imagegame	Use row/column/region commands rather than cell-by-cell. Describe the most distinctive patterns first. Always say DONE when finished. Do not issue multiple commands per turn.
matchit	Describe the most distinctive visual features (shape, color, layout). Focus on whether key features match, not minor details. Remember similar images are not identical. Use only the tags the Game Master requests.
matchit_ascii	Describe structural patterns (rows, columns, filled cells) precisely. Focus on whether key patterns match. Remember similar grids are not identical. Use only the tags the Game Master requests.
multimodal_mapworld	Explore systematically. Track visited rooms. Use room pictures to orient yourself. Prefer unvisited directions. Issue exactly one GO command per turn. If walking in circles, write DONE.
multimodal_referencegame	Describe visual features (color, texture, shape, position) unique to the target and absent from the distractors.
privateshared	Track which information has already been revealed. Answer only what is true according to your private knowledge; use ASIDE for meta-commentary. Do not give multiple answers at once.
referencegame	Focus on features that distinguish the target grid from the distractors. Ignore features shared across grids.
taboo	Describe the target using category, function, or appearance. Avoid morphological variants. Keep clues short and unambiguous.
textmapworld	Explore systematically. Track visited rooms. Prefer unvisited directions. Issue exactly one GO command per turn. If walking in circles, write DONE.
wordle	Start with high-frequency letters. Use each guess to eliminate as many letters as possible. Never repeat a letter already marked red. Never repeat a wrong guess.

Table 10: Human-authored strategies injected into the Upfront-Strategy agent’s conversation history each turn.